FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	RAMOS, Nadine Love D.
Section:	AN43
Professor:	Crisola Tan
Date:	November 2, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. **Which system (RBR or CBR) was easier to implement? Why?**
   RBR was easier to implement because it only needed simple IF–THEN conditions. Each rule checks a fact and gives an action when the condition is true. CBR needed more steps, such as storing old cases, comparing similarities, and updating the case base.
2. **Which is more flexible in solving unstructured problems?**
   CBR is more flexible for unstructured problems because it can use similar past experiences even when a new problem is not an exact match. RBR is more limited because it follows rules that must be written in advance.
3. **Which is better for situations with continuously changing data?**
   CBR can adapt more easily when new experiences are added to its case base. However, RBR can also handle changing data when its facts are updated. For this example, CBR is more adaptable because it can retain new cases and reuse them later.
4. **Examples of real-world applications for both.**
   RBR examples include rule-based troubleshooting, simple medical decision-support guidelines, eligibility checking, and weather alert systems. CBR examples include customer support systems that reuse solutions from previous complaints, equipment repair systems that compare past faults, and help desks that search for similar solved problems.

Your Code - Laboratory Tasks ⇩

```
# PART 1 - KNOWLEDGE REPRESENTATION (KR): KNOWLEDGE BASE

# The dictionary stores facts about the current weather.
# The facts are grouped into three categories:
# temperature, weather conditions, and environment.
facts = {
    "temperature": 32,          # degrees Celsius
    "is_raining": False,
    "humidity": 85,              # percent
    "wind_speed": 10,            # kilometers per hour
    "dark_clouds": True,
    "has_thunder": False
}

# PART 2 - RULE-BASED REASONING (RBR)

def rule_hot_weather(knowledge):
    """IF temperature is above 30, THEN advise staying cool."""
    if knowledge["temperature"] > 30:
        return "The weather is hot. Stay hydrated and seek shade."
    return None

def rule_rainy_weather(knowledge):
    """IF it is raining, THEN advise bringing an umbrella."""
    if knowledge["is_raining"]:
        return "It is raining. Bring an umbrella."
    return None

def rule_high_humidity(knowledge):
     """IF humidity is at least 80%, THEN advise extra comfort
precautions."""
    if knowledge["humidity"] >= 80:
        return "Humidity is high. Stay in a well-ventilated area."
    return None

def rule_dark_clouds(knowledge):
     """IF dark clouds are present, THEN advise monitoring the
weather."""
    if knowledge["dark_clouds"]:
```

```
        return "Dark clouds are present. Monitor the weather for
possible rain."
    return None

def rule_thunderstorm(knowledge):
    """IF thunder is detected, THEN advise moving to a safe indoor
place."""
    if knowledge["has_thunder"]:
        return "Thunder is detected. Stay indoors and follow
weather advisories."
    return None

# The inference engine checks every rule against the knowledge
base.
def inference_engine(knowledge):
    rules = [
        rule_hot_weather,
        rule_rainy_weather,
        rule_high_humidity,
        rule_dark_clouds,
        rule_thunderstorm
    ]

    inferred_actions = []
    for rule in rules:
        result = rule(knowledge)
        if result is not None:
            inferred_actions.append(result)

    return inferred_actions

# PART 3 - CASE-BASED REASONING (CBR)

case_base = [
    {
        "problem": {
            "is_raining": True,
            "dark_clouds": True,
            "high_humidity": True,
            "has_thunder": False
        },
        "solution": "Bring an umbrella and monitor the weather."
    },
    {
```

```
        "problem": {
            "is_raining": False,
            "dark_clouds": False,
            "high_humidity": False,
            "has_thunder": False
        },
         "solution": "Enjoy the clear weather and continue normal
activities."
    },
    {
        "problem": {
            "is_raining": True,
            "dark_clouds": True,
            "high_humidity": True,
            "has_thunder": True
        },
          "solution": "Stay indoors and follow official weather
advisories."
    }
]

def similarity(new_problem, old_problem):
    """
    Compare two cases feature by feature.
    Similarity = matching features / total features.
    The result is a value from 0.0 to 1.0.
    """
    features = new_problem.keys()
    if not features:
        return 0.0

    matches = sum(
        1 for feature in features
        if new_problem[feature] == old_problem.get(feature)
    )
    return matches / len(features)

def retrieve_case(new_problem, cases):
    """RETRIEVE: find and return the most similar past case."""
    best_case = None
    best_score = -1

    for case in cases:
        score = similarity(new_problem, case["problem"])
```

```
        if score > best_score:
            best_score = score
            best_case = case

    return best_case, best_score

def reuse_solution(retrieved_case):
    """REUSE: use the solution from the most similar past case."""
    return retrieved_case["solution"]

def revise_solution(proposed_solution):
    """
    REVISE: allow the user to accept the proposed solution
    or enter an adjusted solution. Press Enter to keep it.
    """
    print("\nProposed solution:", proposed_solution)
    revised = input("Revise the solution? Type a new solution or
press Enter to keep it: ")
    if revised.strip():
        return revised.strip()
    return proposed_solution

def retain_case(new_problem, final_solution, cases):
    """RETAIN: save the new problem and final solution as a new
case."""
    cases.append({
        "problem": new_problem.copy(),
        "solution": final_solution
    })

# MAIN PROGRAM – DISPLAY INPUT, PROCESS, AND OUTPUT

def main():
    print("=" * 60)
    print("TA4: KNOWLEDGE REPRESENTATION, RBR, AND CBR")
    print("=" * 60)

    print("\nPART 1 - KNOWLEDGE BASE")
    for fact, value in facts.items():
        print(f"- {fact}: {value}")

    print("\nPART 2 - RULE-BASED REASONING")
    actions = inference_engine(facts)
    print("Rule-Based Reasoning Actions:")
```

```
    if actions:
        for action in actions:
            print("-", action)
    else:
        print("- No rules were triggered.")

    print("\nPART 3 - CASE-BASED REASONING")
    # This sample problem is based on the current weather facts.
    new_problem = {
        "is_raining": facts["is_raining"],
        "dark_clouds": facts["dark_clouds"],
        "high_humidity": facts["humidity"] >= 80,
        "has_thunder": facts["has_thunder"]
    }

    print("New problem:", new_problem)

    # RETRIEVE
    matched_case, score = retrieve_case(new_problem, case_base)
    print(f"Retrieved case similarity: {score:.0%}")

    # REUSE
    proposed_solution = reuse_solution(matched_case)
    print("Reused solution:", proposed_solution)

    # REVISE
    final_solution = revise_solution(proposed_solution)
    print("Final solution:", final_solution)

if __name__ == "__main__":
    main()
```

```
============================================================
TA4: KNOWLEDGE REPRESENTATION, RBR, AND CBR
============================================================

PART 1 - KNOWLEDGE BASE
- temperature: 32
- is_raining: False
- humidity: 85
- wind_speed: 10
- dark_clouds: True
- has_thunder: False

PART 2 - RULE-BASED REASONING
Rule-Based Reasoning Actions:
- The weather is hot. Stay hydrated and seek shade.
- Humidity is high. Stay in a well-ventilated area.
- Dark clouds are present. Monitor the weather for possible rain.

PART 3 - CASE-BASED REASONING
New problem: {'is_raining': False, 'dark_clouds': True, 'high_humidity': True, 'has_thunder': False}
Retrieved case similarity: 75%
Reused solution: Bring an umbrella and monitor the weather.

Proposed solution: Bring an umbrella and monitor the weather.
Revise the solution? Type a new solution or press Enter to keep it:
Final solution: Bring an umbrella and monitor the weather.
PS C:\Users\ndram\OneDrive\Documents\GitHub>
```